

C++ - Modül 06

C++ Cast'leri

Özet: Bu belge, C++ modüllerinin Modül 06'sına ait alıştırmaları içermektedir.

Sürüm: 8.1

İçindekiler

- I. Giriş
- II. Genel Kurallar
- III. Ek Kural
- IV. Yapay Zeka Talimatları
- V. Alıştırma 00: Scalar type dönüşümü
- VI. Alıştırma 01: Serialization
- VII. Alıştırma 02: Gerçek type'ı belirleme
- VIII. Teslim ve Peer Değerlendirme

Bölüm I — Giriş

C++, Bjarne Stroustrup tarafından C programlama dilinin bir uzantısı olarak oluşturulmuş genel amaçlı bir programlama dilidir; çoğunlukla "C with Classes" olarak anılır (kaynak: Wikipedia).

Bu modüllerin amacı sizi **Object-Oriented Programming**'e tanıştırmaktır. Bu, C++ yolculuğunuzun başlangıç noktası olacaktır. OOP öğrenmek için pek çok dil tavsiye edilse de, C++'ın eski dostunuz C'den türetilmiş olması nedeniyle biz C++'ı seçtik. C++ karmaşık bir dil olduğundan, kodunuz işleri basit tutmak adına C++98 standardına uymak zorundadır.

Modern C++'ın pek çok açıdan önemli ölçüde farklılaştığının farkındayız. Yetkin bir C++ geliştiricisi olmak istiyorsanız, 42 Common Core'un ötesine geçmek size kalmış!

Bölüm II — Genel Kurallar

Derleme

- Kodunuzu `c++` ve `-Wall -Wextra -Werror` flag'leri ile derleyin.
- `-std=c++98` flag'ini eklediğinizde kodunuz yine de derlenebilmelidir.

Biçimlendirme ve isimlendirme kuralları

- Alıştırma dizinleri şu şekilde adlandırılacaktır: `ex00`, `ex01`, ..., `exn`
- Dosyalarınızı, class'larınızı, fonksiyonlarınızı, member fonksiyonlarınızı ve attribute'lerinizi yönergelerde belirtildiği şekilde adlandırın.
- Class isimlerini **UpperCamelCase** formatında yazın. Class kodu içeren dosyalar her zaman class ismine göre adlandırılacaktır. Örneğin:
`ClassName.hpp/ClassName.h`, `ClassName.cpp` veya `ClassName.tpp`.
Dolayısıyla "BrickWall" adlı bir brick wall class'ının tanımını içeren header dosyasının adı `BrickWall.hpp` olacaktır.
- Aksi belirtilmedikçe, her çıktı mesajı bir newline karakteriyle bitmeli ve standart çıktıya yazdırılmalıdır.
- *Norminette'e elveda!* C++ modüllerinde herhangi bir coding style uygulanmaz. Favori stilinizi takip edebilirsiniz. Ancak şunu unutmayın: peer değerlendirmecilerinizin anlayamadığı kod, notlandıramadıkları koddur. Temiz ve okunabilir kod yazmak için elinizden geleni yapın.

İzin Verilenler / Yasaklananlar

Artık C'de kodlamıyorsunuz. C++ zamanı! Bu nedenle:

- Standart kütüphanedeki neredeyse her şeyi kullanabilirsiniz. Dolayısıyla, bildiklerinizle sınırlı kalmak yerine, alıştığınız C fonksiyonlarının C++'a özgü sürümlerini mümkün olduğunca kullanmak akıllıca olacaktır.
- Ancak başka hiçbir harici kütüphane kullanamazsınız. Bu, C++11 (ve türevleri) ile Boost kütüphanelerinin yasak olduğu anlamına gelir. Şu fonksiyonlar da yasaktır: `*printf()`, `*alloc()` ve `free()`. Bunları kullanırsanız notunuz 0 olur, hepsi bu.
- Aksi açıkça belirtilmedikçe `using namespace <ns_name>` ve `friend` keyword'lerinin yasak olduğunu unutmayın. Aksi takdirde notunuz -42 olur.
- **STL'yi yalnızca Modül 08 ve 09'da kullanabilirsiniz.** Bu şu anlama gelir: o zamana kadar **Container** (vector/list/map vb.) ve **Algorithm** (`<algorithm>` header'ını dahil etmeyi gerektiren her şey) kullanılamaz. Aksi takdirde notunuz -42 olur.

Birkaç tasarım gereksinimi

- C++'da da memory leak oluşabilir. Memory tahsis ettiğinizde (`new` keyword'ü kullanarak) **memory leak**'lerden kaçınmak zorundasınız.
- Modül 02'den Modül 09'a kadar, aksi açıkça belirtilmedikçe class'larınız **Orthodox Canonical Form** ile tasarlanmış olmalıdır.
- Bir header dosyasına konan herhangi bir fonksiyon implementasyonu (function template'ler hariç) alıştırma için 0 anlamına gelir.
- Her header dosyanızı diğerlerinden bağımsız olarak kullanabilmeniz gerekir. Dolayısıyla, ihtiyaç duydukları tüm dependency'leri içermeleri gerekir. Ancak **include guard** ekleyerek double inclusion sorununu önlemeniz gerekir. Aksi takdirde notunuz 0 olur.

Beni okuyun

- Gerekirse ek dosyalar ekleyebilirsiniz (örneğin kodunuzu bölmek için). Bu ödevler bir program tarafından doğrulanmadığından, zorunlu dosyaları teslim ettiğiniz sürece bunu yapmakta özgürsünüz.
- Bazen bir alıştırmanın yönergeleri kısa görünebilir ancak örnekler, talimatlarda açıkça yazılmayan gereksinimleri gösterebilir.
- Başlamadan önce her modülü tamamen okuyun! Gerçekten, yapın bunu.
- Odin adına, Thor adına! Beyninizi kullanın!!!

⚠ C++ projeleri için Makefile konusunda, C'deki ile aynı kurallar geçerlidir (Norm'un Makefile bölümüne bakın).

💡 Pek çok class implement etmek zorunda kalacaksınız. Favori metin editörünüzü script'leyebilmedikçe bu sıkıcı görünebilir.

ℹ Alıştırmaları tamamlama konusunda belirli bir özgürlük tanınmaktadır. Ancak zorunlu kurallara uyun ve tembel olmayın. Çok sayıda faydalı bilgiyi kaçırsınız! Teorik kavramlar hakkında okumaktan çekinmeyin.

Bölüm III — Ek Kural

Aşağıdaki kural modülün tamamı için geçerlidir ve zorunludur.

Her alıştırma için, type dönüşümü belirli bir casting türü kullanılarak yapılmalıdır. Tercihiniz savunma sırasında gözden geçirilecektir.

Bölüm IV — Yapay Zeka Talimatları

Bağlam

Bu proje, 42 eğitiminizin temel yapı taşlarını keşfetmenize yardımcı olmak için tasarlanmıştır.

Temel bilgi ve becerileri doğru şekilde pekiştirmek için yapay zeka araçlarını ve desteğini kullanma konusunda düşünceli bir yaklaşım benimsemek çok önemlidir.

Gerçek anlamda temel öğrenme, zorluk, tekrar ve peer-learning alışverişleri aracılığıyla gerçek bir zihinsel çaba gerektirir.

Yapay zekaya yaklaşımımıza — bir öğrenme aracı olarak, 42 eğitiminin bir parçası olarak ve iş piyasasında bir beklenti olarak — daha kapsamlı bir genel bakış için lütfen intranet'teki ilgili FAQ'a başvurun.

Ana mesaj

- 📌 Kısa yollar olmadan güçlü temeller inşa edin.
- 📌 Gerçek anlamda teknik ve güç becerilerini geliştirin.

- Gerçek peer-learning deneyimi yaşayın, öğrenmeyi ve yeni sorunları çözmeyi öğrenmeye başlayın.
- Öğrenme yolculuğu sonuçtan daha önemlidir.
- Yapay zekayla ilişkili riskler hakkında bilgi edinin ve yaygın tuzaklardan kaçınmak için etkili kontrol pratikleri ve karşı önlemler geliştirin.

Öğrenci kuralları

- Özellikle yapay zekaya başvurmadan önce, size verilen görevlere akıl yürüterek yaklaşmalısınız.
- Yapay zekadan doğrudan cevap istememelisiniz.
- 42'nin yapay zekaya küresel yaklaşımını öğrenmelisiniz.

Aşama çıktıları

Bu temel aşamada aşağıdaki çıktıları elde edeceksiniz:

- Uygun teknik ve kodlama temelleri edinme.
- Bu aşamada yapay zekanın neden ve nasıl tehlikeli olabileceğini öğrenme.

Yorumlar ve örnek

- Evet, yapay zekanın var olduğunu biliyoruz — ve evet, projelerinizi çözebilir. Ama burada öğrenmek için buradasınız, yapay zekanın öğrendiğini kanıtlamak için değil. Yapay zekanın verilen problemi çözebildiğini göstermek için zamanınızı (ya da bizimkini) boşa harcamayın.
- 42'de öğrenmek cevabı bilmekle ilgili değil — bir tane bulma yeteneği geliştirmekle ilgilidir. Yapay zeka size cevabı doğrudan verir, ancak bu sizin kendi akıl yürütmenizi oluşturmanızı engeller. Akıl yürütme zaman, çaba gerektirir ve başarısızlığı da kapsar. Başarıya giden yolun kolay olması gerekmiyor.
- Sınavlarda yapay zekanın mevcut olmadığını unutmayın — internet yok, akıllı telefon yok, vb. Öğrenme sürecinizde yapay zekaya çok fazla güvendiyseniz bunu hızla fark edeceksiniz.
- Peer learning sizi farklı fikir ve yaklaşımlara maruz bırakır, kişilerarası becerilerinizi ve ayrıklayıcı düşünme yeteneğinizi geliştirir. Bu, bir botla sohbet etmekten çok daha değerlidir. Çekingen olmayın — konuşun, soru sorun ve birlikte öğrenin!
- Evet, yapay zeka müfredatın bir parçası olacak — hem bir öğrenme aracı olarak hem de başlı başına bir konu olarak. Hatta kendi yapay zeka yazılımınızı oluşturma şansına bile sahip olacaksınız. İntranet'teki belgeler aracılığıyla geçeceğiniz kademeli yaklaşımımız hakkında daha fazla bilgi edinmek için.

✓ İyi pratik:

Yeni bir konseptte takılıp kaldım. Yakınımdaki birine nasıl yaklaştığımı soruyorum. 10 dakika konuşuyoruz — ve bir anda anlıyor. Kavrardım.

✗ Kötü pratik:

Gizlice yapay zeka kullanıyorum, doğru görünen bir kod kopyalıyorum. Peer değerlendirmesi sırasında hiçbir şeyi açıklayamıyorum. Başarısız oluyorum. Sınavda — yapay zeka yok — yine takılıp kalıyorum. Başarısız oluyorum.

Bölüm V — Alıştırma 00: Scalar Type Dönüşümü

Alıştırma 00

Scalar type dönüşümü

Dizin: `ex00/`

Teslim edilecek dosyalar: `Makefile, *.cpp, *.{h, hpp}`

İzin verilenler: Bir string'den int, float veya double'a dönüştürmek için herhangi bir fonksiyon. Bu yardımcı olacaktır ancak işin tamamını yapmayacaktır.

Yalnızca tek bir `static` method "convert" içerecek olan `ScalarConverter` adlı bir class yazın; bu method, bir C++ literal'ının en yaygın formundaki string temsilini parametre olarak alacak ve değerini aşağıdaki scalar type serisi biçiminde çıktı verecektir:

- `char`
- `int`
- `float`
- `double`

Bu class'ın herhangi bir şey depolaması gerekmediğinden, kullanıcılar tarafından örneklendirilememesi gerekir. `char` parametreleri dışında yalnızca ondalık gösterim kullanılacaktır.

`char` literal örnekleri: `'c'`, `'a'`, ...

İşleri basit tutmak için, non-displayable karakterlerin input olarak kullanılmaması gerektiğini unutmayın. `char`'a dönüşüm görüntülenemiyorsa, bilgilendirici bir mesaj yazdırın.

`int` literal örnekleri: `0`, `-42`, `42`...

`float` literal örnekleri: `0.0f`, `-4.2f`, `4.2f`...

Bu pseudo-literal'ları da işlemeniz gerekir (bilirsiniz, bilim adına): `-inff`, `+inff` ve `nanf`.

`double` literal örnekleri: `0.0`, `-4.2`, `4.2`...

Bu pseudo-literal'ları da işlemeniz gerekir (bilirsiniz, eğlence olsun diye): `-inf`, `+inf` ve `nan`.

Class'ınızın beklendiği gibi çalıştığını test etmek için bir program yazın.

Önce parametre olarak geçirilen literal'ın type'ını tespit etmeli, onu string'den gerçek type'ına dönüştürmeli, ardından diğer üç veri type'ına **açıkça** dönüştürmelisiniz. Son olarak sonuçları aşağıda gösterildiği gibi görüntüleyin.

Bir dönüşüm mantıklı değilse veya overflow oluyorsa, kullanıcıyı type dönüşümünün imkânsız olduğu konusunda bilgilendiren bir mesaj görüntüleyin. Numeric limit'leri ve özel değerleri işlemek için ihtiyacınız olan herhangi bir header'ı dahil edin.

```
./convert 0
char: Non displayable
int: 0
float: 0.0f
double: 0.0
```

```
./convert nan
char: impossible
int: impossible
float: nanf
double: nan
```

```
./convert 42.0f
char: '*'
int: 42
float: 42.0f
double: 42.0
```

Bölüm VI — Alıştırma 01: Serialization

Alıştırma 01

Serialization

Dizin: `ex01/`

Teslim edilecek dosyalar: `Makefile, *.cpp, *.{h, hpp}`

Yasaklananlar: Yok

Kullanıcı tarafından hiçbir şekilde başlatılamayacak olan `Serializer` adlı bir class implement edin; bu class aşağıdaki `static` method'lara sahip olmalıdır:

```
cpp
uintptr_t serialize(Data* ptr);
```

Bir pointer alır ve onu `uintptr_t` unsigned integer type'ına dönüştürür.

```
cpp
```

```
Data* deserialize(uintptr_t raw);
```

Bir unsigned integer parametre alır ve onu `Data`'ya pointer'a dönüştürür.

Class'ınızın beklediği gibi çalıştığını test etmek için bir program yazın.

Non-empty (yani data member'ları olan) bir `Data` structure'ı oluşturmanız gerekir.

`Data` nesnesinin adresine `serialize()` uygulayın ve dönüş değerini `deserialize()`'a geçirin. Ardından `deserialize()`'ın dönüş değerinin orijinal pointer'a eşit olduğunu doğrulayın.

`Data` structure'ınızın dosyalarını teslim etmeyi unutmayın.

Bölüm VII — Alıştırma 02: Gerçek Type'ı Belirleme

Alıştırma 02

Gerçek type'ı belirleme

Dizin: `ex02/`

Teslim edilecek dosyalar: `Makefile, *.cpp, *.{h, hpp}`

Yasaklananlar: `std::typeinfo`

Yalnızca public virtual destructor'a sahip bir `Base` class implement edin. `Base`'den public olarak miras alan üç boş class oluşturun: `A`, `B` ve `C`.

 Bu dört class Orthodox Canonical Form ile tasarlanmak zorunda değildir.

Aşağıdaki fonksiyonları implement edin:

```
cpp
Base* generate(void);
```

`A`, `B` veya `C`'yi rastgele örneklendirir ve instance'ı bir `Base` pointer'ı olarak döndürür. Random seçim implementasyonu için istediğiniz her şeyi kullanmakta serbestsiniz.

```
cpp
void identify(Base* p);
```

`p` tarafından işaret edilen nesnenin gerçek type'ını yazdırır: `"A"`, `"B"` veya `"C"`.

```
cpp
void identify(Base& p);
```

`p` tarafından referans verilen nesnenin gerçek type'ını yazdırır: `"A"`, `"B"` veya `"C"`. Bu fonksiyonun içinde pointer kullanmak yasaktır.

`typeinfo` header'ını dahil etmek yasaktır.

Her şeyin beklendiği gibi çalıştığını test etmek için bir program yazın.

Bölüm VIII — Teslim ve Peer Değerlendirme

Ödevi her zamanki gibi Git repository'nize gönderin. Savunma sırasında yalnızca repository'nizin içindeki çalışma değerlendirilecektir. Klasör ve dosya isimlerinin doğru olduğundan emin olmak için tekrar kontrol etmekten çekinmeyin.

Değerlendirme sırasında, projenin kısa bir **modifikasyonu** zaman zaman istenebilir. Bu, küçük bir davranış değişikliği, yazılacak veya yeniden yazılacak birkaç satır kod ya da kolayca eklenebilecek bir özellik içerebilir.

Bu adım her proje için geçerli **olmayabilir**, ancak değerlendirme yönergelerinde belirtilmişse bunun için hazırlıklı olmanız gerekir.

Bu adım, projenin belirli bir bölümünü gerçekten anlayıp anlamadığınızı doğrulamak amacıyla tasarlanmıştır. Modifikasyon, seçtiğiniz herhangi bir geliştirme ortamında yapılabilir (örneğin, alıştığınız kurulum) ve değerlendirmenin bir parçası olarak belirli bir zaman dilimi tanımlanmadığı sürece birkaç dakika içinde yapılabilir olmalıdır.

Örneğin, bir fonksiyon veya script'te küçük bir güncelleme yapmanız, bir görüntüyü değiştirmeniz ya da yeni bilgileri depolamak için bir data structure'ı düzenlemeniz istenebilir vb.

Ayrıntılar (kapsam, hedef vb.) **değerlendirme yönergelerinde** belirtilecektir ve aynı proje için bir değerlendirmeden diğerine farklılık gösterebilir.